

Introduction à la complexité algorithmique

Comparaison recherche séquentielle / recherche dichotomique — avec preuve, exercice pratique et étude de cas Google

1. Pourquoi mesurer la complexité ?

La complexité d'un algorithme mesure le nombre d'opérations qu'il effectue en fonction de la taille de l'entrée (notée n). L'objectif n'est pas de compter les opérations exactement, mais d'estimer comment le temps de calcul **évolue** quand n grandit — c'est la notation **$O(\dots)$** (grand O).

2. Recherche séquentielle

On cherche une valeur x dans un tableau de taille n , en parcourant les éléments un par un.

```
def recherche_sequentielle(tab, x):
    for i in range(len(tab)):
        if tab[i] == x:
            return i
    return -1
```

- **Pire cas** : x n'est pas dans le tableau (ou est en dernière position) $\rightarrow n$ comparaisons.
- **Meilleur cas** : x est en première position $\rightarrow 1$ comparaison.
- **Complexité au pire cas** : **$O(n)$** — linéaire.

Aucune hypothèse n'est faite sur l'ordre du tableau : ça marche même si `tab` n'est pas trié.

3. Recherche dichotomique (binaire)

Ici, le tableau doit être **trié**. On compare x à l'élément du milieu, puis on élimine la moitié du tableau à chaque étape.

```
def recherche_dichotomique(tab, x):
    gauche, droite = 0, len(tab) - 1
    while gauche <= droite:
        milieu = (gauche + droite) // 2
        if tab[milieu] == x:
            return milieu
        elif tab[milieu] < x:
            gauche = milieu + 1
        else:
            droite = milieu - 1
    return -1
```

À chaque itération, la taille de l'espace de recherche est divisée par 2. On se demande alors : combien de fois peut-on diviser n par 2 avant d'arriver à 1 ? C'est exactement la définition du logarithme en base 2 : **$O(\log n)$** .

Comparaison concrète

Taille du tableau (n)	Séquentielle (pire cas)	Dichotomique (pire cas)
10	10	~4
1 000	1 000	~10
1 000 000	1 000 000	~20
1 000 000 000	1 000 000 000	~30

L'écart devient spectaculaire : pour un milliard d'éléments, la dichotomique fait en **30 comparaisons** ce que la séquentielle fait en **un milliard**.

Le point pédagogique clé

1. **Structurer les données a un coût caché mais rentable** — trier le tableau a un prix, mais rend ensuite la recherche bien plus rapide.
2. **La notation $O(\dots)$ capture la croissance, pas la vitesse absolue** — $O(\log n)$ reste bien plus performant que $O(n)$ même si les constantes cachées diffèrent.
3. **Le pire cas est la référence standard** en algorithmique, car il garantit une borne de performance quel que soit l'entrée.

4. Preuve formelle du $O(\log n)$

Objectif : montrer que la recherche dichotomique effectuée au plus $\lfloor \log_2(n) \rfloor + 1$ comparaisons dans le pire cas, sur un tableau de taille n .

Mise en place

Notons $T(n)$ le nombre de comparaisons effectuées dans le pire cas pour un tableau de taille n . À chaque appel :

- On fait **1 comparaison** (`tab[milieu] vs x`).
- Si ce n'est pas une égalité, on relance la recherche sur un sous-tableau de taille au plus $n/2$.

Cela donne la relation de récurrence :

$$T(n) = T(n/2) + 1, \quad T(1) = 1$$

Résolution par substitution

On pose $n = 2^k$ pour simplifier (le raisonnement se généralise ensuite à tout n en encadrant par la puissance de 2 la plus proche).

$$T(2^k) = T(2^{(k-1)}) + 1$$

En dépliant la récurrence :

$$T(2^k) = T(2^{(k-2)}) + 2 = T(2^{(k-3)}) + 3 = \dots = T(2^0) + k = 1 + k$$

Or $k = \log_2(n)$, donc :

$$T(n) = \log_2(n) + 1$$

Conclusion

Pour un n quelconque (pas forcément une puissance de 2), on montre par encadrement que :

$$T(n) \leq \lfloor \log_2(n) \rfloor + 1$$

Ce qui s'écrit en notation de complexité : **$T(n) = O(\log n)$** .

Intuition pour la classe : c'est la question « combien de fois faut-il diviser n par 2 pour arriver à 1 ? » — c'est précisément la définition du logarithme en base 2. On peut aussi faire le lien visuel avec un arbre binaire de profondeur $\log_2(n)$.

5. Exercice pratique (mesure expérimentale)

Objectif : faire compter aux élèves le nombre réel de comparaisons, tracer la courbe, et observer la différence de croissance.

```
import random
import matplotlib.pyplot as plt

def recherche_sequentielle(tab, x):
    comparaisons = 0
    for i in range(len(tab)):
        comparaisons += 1
        if tab[i] == x:
            return comparaisons
    return comparaisons

def recherche_dichotomique(tab, x):
    comparaisons = 0
    gauche, droite = 0, len(tab) - 1
    while gauche <= droite:
        comparaisons += 1
        milieu = (gauche + droite) // 2
        if tab[milieu] == x:
            return comparaisons
        elif tab[milieu] < x:
            gauche = milieu + 1
        else:
            droite = milieu - 1
    return comparaisons
```

```
# Experience : pire cas (element absent) pour differentes tailles
tailles = [10, 100, 1000, 10000, 100000, 1000000]
resultats_seq = []
resultats_dicho = []

for n in tailles:
    tab = list(range(n))      # tableau trie 0, 1, ..., n-1
    x = -1                    # valeur absente -> force le pire cas
    resultats_seq.append(recherche_sequentielle(tab, x))
    resultats_dicho.append(recherche_dichotomique(tab, x))

# Affichage des resultats
for n, s, d in zip(tailles, resultats_seq, resultats_dicho):
    print(f"n={n:>8} | sequentielle: {s:>8} | dichotomique: {d:>3}")

# Trace graphique
plt.plot(tailles, resultats_seq, marker='o', label='Sequentielle O(n)')
plt.plot(tailles, resultats_dicho, marker='o', label='Dichotomique O(log n)')
plt.xscale('log')
plt.xlabel('Taille du tableau (n)')
plt.ylabel('Nombre de comparaisons')
plt.legend()
plt.title('Comparaisons : sequentielle vs dichotomique')
plt.show()
```

Questions à poser aux élèves après l'exécution

1. Que remarquez-vous sur la colonne « dichotomique » quand n est multiplié par 10 ? (*Réponse attendue : elle n'augmente que de quelques unités — cohérent avec $\log_{10}(10) \approx 3,3$ divisions supplémentaires en base 2.*)
2. Pourquoi utilise-t-on une échelle logarithmique (`xscale('log')`) sur l'axe des n ?
3. Si on cherchait un élément **présent** au hasard (au lieu du pire cas), la courbe de la séquentielle changerait-elle beaucoup ? Et celle de la dichotomique ?
4. **Bonus** : modifier le code pour mesurer le **temps réel** (`time.perf_counter()`) plutôt que le nombre de comparaisons — les élèves verront que le temps mesuré est bruité par d'autres facteurs (cache, interpréteur Python), contrairement au compte de comparaisons qui est une mesure « pure » de la complexité.

6. Troisième exemple : l'algorithme de recherche de Google

Cet exemple est intéressant en classe parce qu'il montre que la complexité ne se limite pas aux tableaux — c'est un problème sur des **graphes**, à une échelle où la naïveté est impossible.

Le problème

Google doit répondre à une recherche parmi des **milliards** de pages web, en une fraction de seconde. Deux défis distincts se posent :

1. **Trouver les pages contenant les mots-clés** — un problème de recherche.
2. **Classer les résultats par pertinence** — c'est là qu'intervient **PageRank**, l'algorithme historique de Google.

Pourquoi la recherche séquentielle est impossible ici

Si Google devait parcourir chaque page du web à chaque requête pour vérifier si elle contient les mots recherchés, ce serait une recherche séquentielle sur des **milliards** de documents — totalement irréalisable en temps réel.

Solution : l'index inversé. Au lieu de parcourir les pages à chaque recherche, Google construit à l'avance une structure qui associe à chaque mot la liste des pages où il apparaît :

```
# Index inverse simplifie
index = {
    "python": ["page1", "page5", "page42", ...],
    "algorithme": ["page3", "page5", "page99", ...],
}
```

Chercher les pages contenant « python » devient alors une simple lecture dans un dictionnaire (table de hachage), en **O(1)** en moyenne, au lieu de $O(n)$ sur tout le web. C'est le même principe pédagogique que la dichotomique : **payer un coût de structuration à l'avance pour rendre chaque requête ultérieure quasi instantanée.**

PageRank : classer par importance

Une fois les pages candidates trouvées, il faut les classer. L'idée de PageRank (Larry Page & Sergey Brin, 1998) : une page est

importante si elle est pointée par beaucoup d'autres pages importantes — un calcul récursif sur le graphe du web.

Concrètement, PageRank calcule un vecteur de scores en itérant une formule sur toutes les pages jusqu'à convergence (méthode de la puissance itérée sur une matrice). Pour un graphe à n pages et m liens, chaque itération coûte environ **$O(n + m)$** , répétée un petit nombre fixe de fois — donc **quasi-linéaire** en la taille du web, ce qui est remarquable vu l'échelle.

Le point pédagogique

Approche naïve	Approche réelle de Google
Parcourir toutes les pages à chaque recherche	Index inversé construit à l'avance
Complexité par requête : $O(n)$ sur des milliards de pages	Complexité par requête : $\sim O(1)$ à $O(\log n)$
Classement improvisé	PageRank : calcul de graphe en $O(n + m)$

Ce troisième exemple permet de faire la transition entre « structures de données simples » (tableaux triés) et « structures de données réelles » (index, graphes), et montre aux élèves que **la complexité algorithmique est ce qui rend possible, ou impossible, un produit comme Google à l'échelle industrielle.**